

In Angular with **RxJS**, if you want to **wait until a variable (observable) matches a certain value** before continuing, you can use operators like:

- `filter()` → only lets through values that match your condition.
- `first()` → completes after the first matching value.
- `take(1)` → similar to `first()` but without throwing if no match.
- `skipWhile()` → skip until condition is false.

Example: Wait until a value becomes `true` once

```
import { BehaviorSubject, filter, first } from 'rxjs';

// Example observable variable
const isReady$ = new BehaviorSubject<boolean>(false);

// Somewhere in your code: wait until it's true
isReady$
  .pipe(
    filter(value => value === true), // only pass when true
    first() // complete after first match
  )
  .subscribe(() => {
    console.log('Value is now true, continuing...');
    // Place your logic here
  });

// Simulate async change
```

```
setTimeout(() => {  
  isReady$.next(true);  
}, 2000);
```

Example: React every time the value matches

```
import { BehaviorSubject, filter } from 'rxjs';  
  
const status$ = new BehaviorSubject<string>('loading');  
  
status$  
  .pipe(  
    filter(status => status === 'ready') // only when ready  
  )  
  .subscribe(() => {  
    console.log('Status is ready!');  
  });  
  
// Simulate changes  
setTimeout(() => status$.next('ready'), 3000);  
setTimeout(() => status$.next('ready'), 5000); // triggers again
```

Key Points

- `filter()` ensures only matching values pass through.
- `first()` or `take(1)` ensures you only wait for the first match and then complete.

- Works with any observable source (BehaviorSubject, ReplaySubject, fromEvent, HTTP calls, etc.).
- If you need to **block** until a condition is met inside an async function, you can await the observable by converting it to a promise:

```
import { firstValueFrom } from 'rxjs';

await firstValueFrom(
  isReady$.pipe(filter(v => v === true))
);
console.log('Ready!');
```

If you want, I can give you a **version that works directly inside Angular services/components** so it integrates cleanly with lifecycle hooks. Do you want me to prepare that?